Experiment No. 1

Aim: Write a JavaScript program to demonstrate the scope and mutability of var, let, and const.

Concept Used: JavaScript is used to demonstrate variable scope and mutability concepts. The code utilizes var, let, and const declarations to showcase their differences. Var has function scope, while let and const have block scope. This affects their mutability and accessibility.

Code:

```
var x = 10;
let y = 20;
const z = 30;
console.log("Outside: ", x, y, z);
if (true) {
   var x = 40;
   let y = 50;
   const z = 60;
   console.log("Inside: ", x, y, z);
}
console.log("Outside: ", x, y, z);
```

Output:

Figure 1 - Scope and Mutability of var, let & const Output

Experiment No. 2

Aim: Write a JavaScript program to demonstrate the usage of primitive data types and the typeof operator.

Concept Used: The program utilizes JavaScript's primitive data types, including number, string, boolean, null, and undefined. The typeof operator is used to determine the data type of a variable. This demonstrates the basic data types and operator usage in JavaScript. Type checking is performed.

Code:

```
let num = 10;
let str = "Hello";
let flag = true;
let empty = null;
let undef;
console.log(typeof num);
console.log(typeof str);
console.log(typeof flag);
console.log(typeof empty);
console.log(typeof undef);
```

Output:

Figure 2 - JavaScript typeof Output

Experiment No. 3

Aim: Write a JavaScript program to evaluate expressions using comparison, logical, and ternary operators.

Concept Used: The program utilizes JavaScript comparison, logical, and ternary operators to evaluate expressions. It employs if-else statements and conditional expressions to demonstrate these operators. The code showcases the usage of ==, !=, &&, ||, and ?: operators. It also highlights the importance of operator precedence.

Code:

```
let x = 10;
let y = 5;
console.log("Comparison Operators:");
console.log(x > y);
console.log(x < y);
console.log(x == y);
console.log(x != y);
console.log("Logical Operators:");
console.log(x > y && x == 10);
console.log(x > y || x == 5);
console.log("Ternary Operator:");
console.log(x > y ? "x is greater" : "y is greater");
```

Output:

Figure 3 - JavaScript Operators Output

Experiment No. 4

Aim: Write a JavaScript program to implement control flow using if-else and switch statements.

Concept Used: Control flow is managed using if-else and switch statements in JavaScript programming. Conditional statements are utilized for decision-making. These statements execute different blocks of code based on specific conditions. JavaScript syntax is used for implementation.

Code:

```
const num = 2;
if (num === 1) {
   console.log("Number is 1");
} else if (num === 2) {
   console.log("Number is 2");
} else {
   console.log("Number is not 1 or 2");
}
const day = "Monday";
switch (day) {
   case "Monday":
      console.log("Today is Monday");
      break;
   case "Tuesday":
      console.log("Today is Tuesday");
      break;
   default:
      console.log("Today is not Monday or Tuesday");
}
```

Output:

Figure 4 - Control Flow Output

Experiment No. 5

Aim: Write a JavaScript program to implement various looping statements including for, while, do-while, for-in, and for-of.

Concept Used: The program utilizes JavaScript looping statements, including for, while, do-while, for-in, and for-of, to iterate over arrays and objects. These statements allow for repeated execution of code and are essential in programming. JavaScript is the chosen language due to its versatility and extensive support for various looping constructs.

Code:

```
var fruits = ["apple", "banana", "cherry"];
for (var i = 0; i < fruits.length; i++) {
   console.log(fruits[i]);
}
console.log("");

var i = 0;
while (i < fruits.length) {
   console.log(fruits[i]);
   i++;
}
console.log("");

var i = 0;
do {
   console.log(fruits[i]);
   i++;
} while (i < fruits.length);
console.log("");

// Object for for-in loop

var person = {
   name: "John",
   age: 30,
   city: "New York"
};
```

```
// FOR-IN LOOP

for (var key in person) {
   console.log(key + ": " + person[key]);
}
console.log("");

// FOR-OF LOOP

for (var fruit of fruits) {
   console.log(fruit);
}
```

Output:

Figure 5 - Looping Output

Experiment No. 6

Aim: Write a JavaScript program to demonstrate object creation, property access, and dynamic properties.

Concept Used: The programming concepts used include object-oriented programming, dynamic property creation, and property access. JavaScript is utilized for its flexibility in handling objects. Object creation and manipulation are demonstrated through property access and dynamic property assignment.

Code:

```
let person = { name: "John", age: 30 };
console.log(person.name);
person.occupation = "Developer";
console.log(person);
delete person.age;
console.log(person);
```

Output:

Figure 6 - Object Creation

Experiment No. 7

Aim: Write a JavaScript program to demonstrate the difference between function declaration and function expression, focusing on hoisting behavior.

Concept Used: Hoisting is a mechanism in JavaScript where variable and function declarations are moved to the top of their containing scope during compilation. Function declarations are fully hoisted, meaning they are initialized with their function object before any code execution, while function expressions are not hoisted in the same way and must be evaluated when encountered.

Code:

```
function declaredFunction() { console.log("This is a declared function"); }
var expressedFunction = function() { console.log("This is an expressed function"); }
console.log(typeof declaredFunction);
console.log(typeof expressedFunction);
console.log(typeof undeclaredFunction);
var undeclaredFunction = function() { console.log("This is an undeclared expressed
function"); }
```

Output:

Figure 7 - Hoisting Output

Experiment No. 8

Aim: Write a JavaScript program to demonstrate the usage of arrow functions and the lexical "this" concept.

Concept Used: The program utilizes arrow functions and lexical this concept, demonstrating how arrow functions inherit the this context from their surroundings. This is a key concept in JavaScript, allowing for more intuitive and flexible function definitions. It enables the creation of concise and readable code. Lexical scoping is also applied.

Code:

```
const obj = {
  name: 'John',
  sayName: () => {
    console.log(this.name);
  },
  sayNameRegular: function() {
    console.log(this.name);
  }
};
const obj2 = {
  name: 'Jane',
  sayName: () => {
    console.log(this.name);
  },
  sayNameRegular: function() {
    console.log(this.name);
  }
};
obj.sayName();
obj.sayNameRegular();
obj2.sayName();
obj2.sayNameRegular();
```

Output:

Figure 8 - Arrow Function Output

Experiment No. 9

Aim: Write a JavaScript program to handle function arguments using default parameters and the rest operator.

Concept Used: Default parameters allow a function in JavaScript to assign predefined values when arguments are not provided or are passed as undefined. The rest operator (...) collects multiple remaining arguments into a single array, making it convenient to process variable-length input. Together, these features improve flexibility and readability in function design. This implementation uses JavaScript to demonstrate both fixed and variable argument handling.

Code:

```
function greet(name = "Student", course = "JavaScript Lab") {
   return `Hello, ${name}! Welcome to ${course}.`;
}

function sumNumbers(label = "Sum", ...numbers) {
   let total = 0;
   for (let num of numbers) {
      total += num;
   }
   return `${label}: ${total}`;
}

function showItems(title = "Items", ...items) {
   if (items.length === 0) {
      return `${title}: No items provided`;
   }
   return `${title}: ${items.join(", ")}`;
}

console.log("Function Arguments Demo");
console.log("----------------------");

console.log(greet());
```

```
console.log(greet("Anjali"));
console.log(greet("Rahul", "Web Development"));

console.log(sumNumbers());
console.log(sumNumbers("Marks", 78, 85, 90));
console.log(sumNumbers("Expenses", 1200, 450, 800, 150));

console.log(showItems());
console.log(showItems("Fruits", "Apple", "Banana", "Orange"));
console.log(showItems("Languages", "JavaScript", "Python", "Java"));
```

Output:

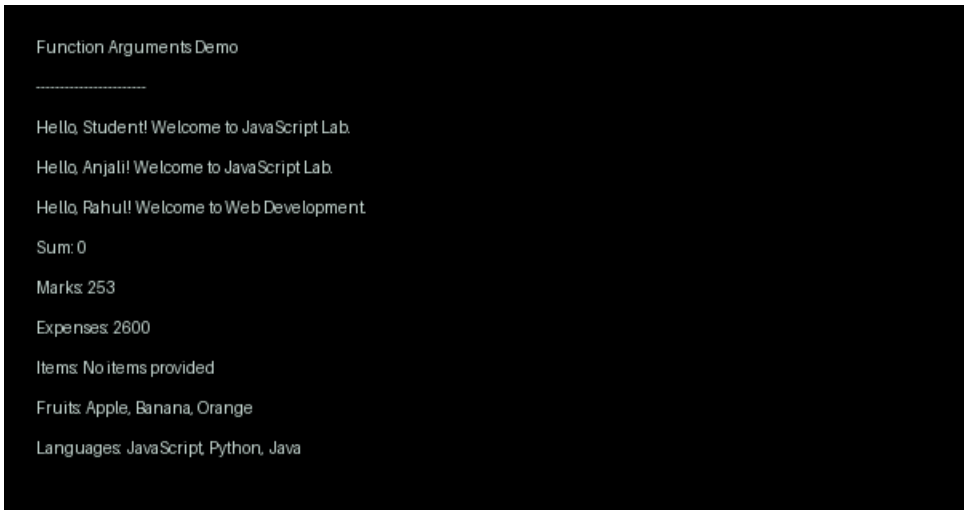

Figure 9 - Default and rest demo

Experiment No. 10

Aim: Write a JavaScript program to demonstrate call by value and call by reference behaviors.

Concept Used: Call by value means a function receives a copy of a variable's value, so changes inside the function do not affect the original variable. In JavaScript, primitive data types such as number, string, and boolean behave this way. For objects and arrays, JavaScript passes a reference value, so modifying object properties inside a function affects the same original object. This implementation uses JavaScript to demonstrate both behaviors clearly.

Code:

```
function modifyPrimitive(x) {
   console.log("Inside modifyPrimitive before change:", x);
   x = x + 10;
   console.log("Inside modifyPrimitive after change :", x);
}

function modifyObject(obj) {
   console.log("Inside modifyObject before change   :", obj);
   obj.value = obj.value + 10;
   console.log("Inside modifyObject after change    :", obj);
}

let num = 25;
let data = { value: 25 };

console.log("Before modifyPrimitive call:", num);
modifyPrimitive(num);
console.log("After modifyPrimitive call :", num);

console.log("");

console.log("Before modifyObject call   :", data);
```

```
modifyObject(data);
console.log("After modifyObject call    :", data);
```

Output:

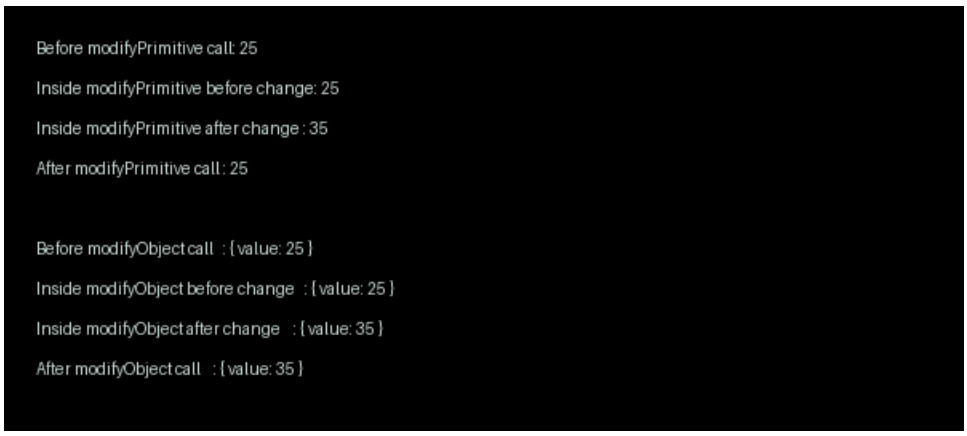

Figure 10 - Value vs reference demo

Experiment No. 11

Aim: Write a JavaScript program to calculate the factorial of a number using recursive functions and lambda expressions.

Concept Used: Factorial is a mathematical operation in which a positive integer n is multiplied by all integers from 1 to n. A recursive function solves this by calling itself with a smaller value until a base condition is reached.
Lambda expressions in JavaScript are implemented using arrow functions, which provide a concise way to define functions. In this implementation, recursion is combined with an arrow function to compute the factorial value.
The program also includes simple input validation to ensure factorial is calculated only for non-negative integers. Thus, the implementation uses JavaScript to demonstrate both recursion and lambda expression syntax.

Code:

```
const number = 5;
const factorial = (n) => {
   if (!Number.isInteger(n) || n < 0) {
      return "Factorial is defined only for non-negative integers.";
   }
   if (n === 0 || n === 1) {
      return 1;
   }
   return n * factorial(n - 1);
};
const result = factorial(number);
console.log("Number:", number);
console.log("Factorial:", result);
```

Output:

Figure 11 - Factorial using recursion

Experiment No. 12

Aim: Write a JavaScript program to implement closures for creating private variables and data encapsulation.

Concept Used: Closures are a fundamental feature in JavaScript where an inner function retains access to variables from its outer lexical scope even after the outer function has finished execution. This property is used to create private variables that cannot be accessed directly from outside the function. Data encapsulation is achieved by exposing only selected methods that can read or modify the hidden state. Thus, this JavaScript implementation demonstrates controlled access, privacy, and modular object-like design.

Code:

```
function createBankAccount(accountHolder, initialBalance) {
  let balance = initialBalance;

  return {
    getAccountHolder: function () {
      return accountHolder;
    },
    getBalance: function () {
      return balance;
    },
    deposit: function (amount) {
      if (amount > 0) {
        balance += amount;
        console.log("Deposited: " + amount);
      } else {
        console.log("Invalid deposit amount");
      }
    },
    withdraw: function (amount) {
      if (amount <= 0) {
        console.log("Invalid withdrawal amount");
      } else if (amount > balance) {
        console.log("Insufficient balance");
```

```
        } else {
            balance -= amount;
            console.log("Withdrawn: " + amount);
        }
    }
  };
}

const account = createBankAccount("Aarav", 5000);

console.log("Account Holder: " + account.getAccountHolder());
console.log("Initial Balance: " + account.getBalance());

account.deposit(1500);
console.log("Balance after deposit: " + account.getBalance());

account.withdraw(2000);
console.log("Balance after withdrawal: " + account.getBalance());

account.withdraw(6000);
account.deposit(-100);

console.log("Final Balance: " + account.getBalance());
console.log("Direct balance access: " + account.balance);
```

Output:

Figure 12 - Closure-based private data

Experiment No. 13

Aim: Write a JavaScript program to demonstrate array manipulation using higher-order functions like map(), filter(), reduce(), and asynchronous execution with setTimeout().

Concept Used: This implementation uses JavaScript to demonstrate array manipulation through higher-order functions. The map() method transforms each element, filter() selects elements based on a condition, and reduce() aggregates values into a single result. The program also uses setTimeout() to illustrate asynchronous execution by delaying a task without blocking the main thread. Together, these features show functional and event-driven programming concepts in JavaScript.

Code:

```
const numbers = [5, 12, 8, 130, 44];

console.log("Original array:", numbers);

const squared = numbers.map(num => num * num);
console.log("Squared values:", squared);

const greaterThanTen = numbers.filter(num => num > 10);
console.log("Values greater than 10:", greaterThanTen);

const sum = numbers.reduce((acc, num) => acc + num, 0);
console.log("Sum of array elements:", sum);

console.log("Starting asynchronous task...");

setTimeout(() => {
  const evenNumbers = numbers.filter(num => num % 2 === 0);
  console.log("Asynchronous output after 2 seconds:");
  console.log("Even numbers:", evenNumbers);

  const doubledEvenNumbers = evenNumbers.map(num => num * 2);
  console.log("Doubled even numbers:", doubledEvenNumbers);
```

```
  const product = evenNumbers.reduce((acc, num) => acc * num, 1);
  console.log("Product of even numbers:", product);
}, 2000);

console.log("Main program continues without waiting for setTimeout.");
```

Output:

Figure 13 - Array methods and async demo

Experiment No. 14

Aim: Write a JavaScript program to demonstrate array and object destructuring, along with JSON parsing and stringification.

Concept Used: Destructuring in JavaScript is a concise syntax used to extract values from arrays and properties from objects into individual variables. Array destructuring assigns values by position, while object destructuring assigns values by matching property names. JSON parsing converts a JSON-formatted string into a JavaScript object, and stringification converts a JavaScript object into a JSON string. This implementation uses JavaScript to demonstrate these concepts in a clear and practical manner.

Code:

```
const student = {
  name: "Aarav",
  age: 20,
  course: "Computer Science",
  marks: {
    math: 92,
    programming: 95
  },
  hobbies: ["Reading", "Coding", "Chess"]
};

const numbers = [10, 20, 30, 40, 50];
const [first, second, ...remaining] = numbers;

const {
  name,
  age,
  course,
  marks: { math, programming },
  hobbies: [hobby1, hobby2, hobby3]
} = student;

const jsonString = JSON.stringify(student, null, 2);
```

```
const parsedObject = JSON.parse(jsonString);

console.log("Array Destructuring:");
console.log("First:", first);
console.log("Second:", second);
console.log("Remaining:", remaining);

console.log("\nObject Destructuring:");
console.log("Name:", name);
console.log("Age:", age);
console.log("Course:", course);
console.log("Math Marks:", math);
console.log("Programming Marks:", programming);
console.log("Hobbies:", hobby1 + ", " + hobby2 + ", " + hobby3);

console.log("\nJSON Stringification:");
console.log(jsonString);

console.log("\nJSON Parsing:");
console.log("Parsed Name:", parsedObject.name);
console.log("Parsed Course:", parsedObject.course);
console.log("Parsed Programming Marks:", parsedObject.marks.programming);
```

Output:

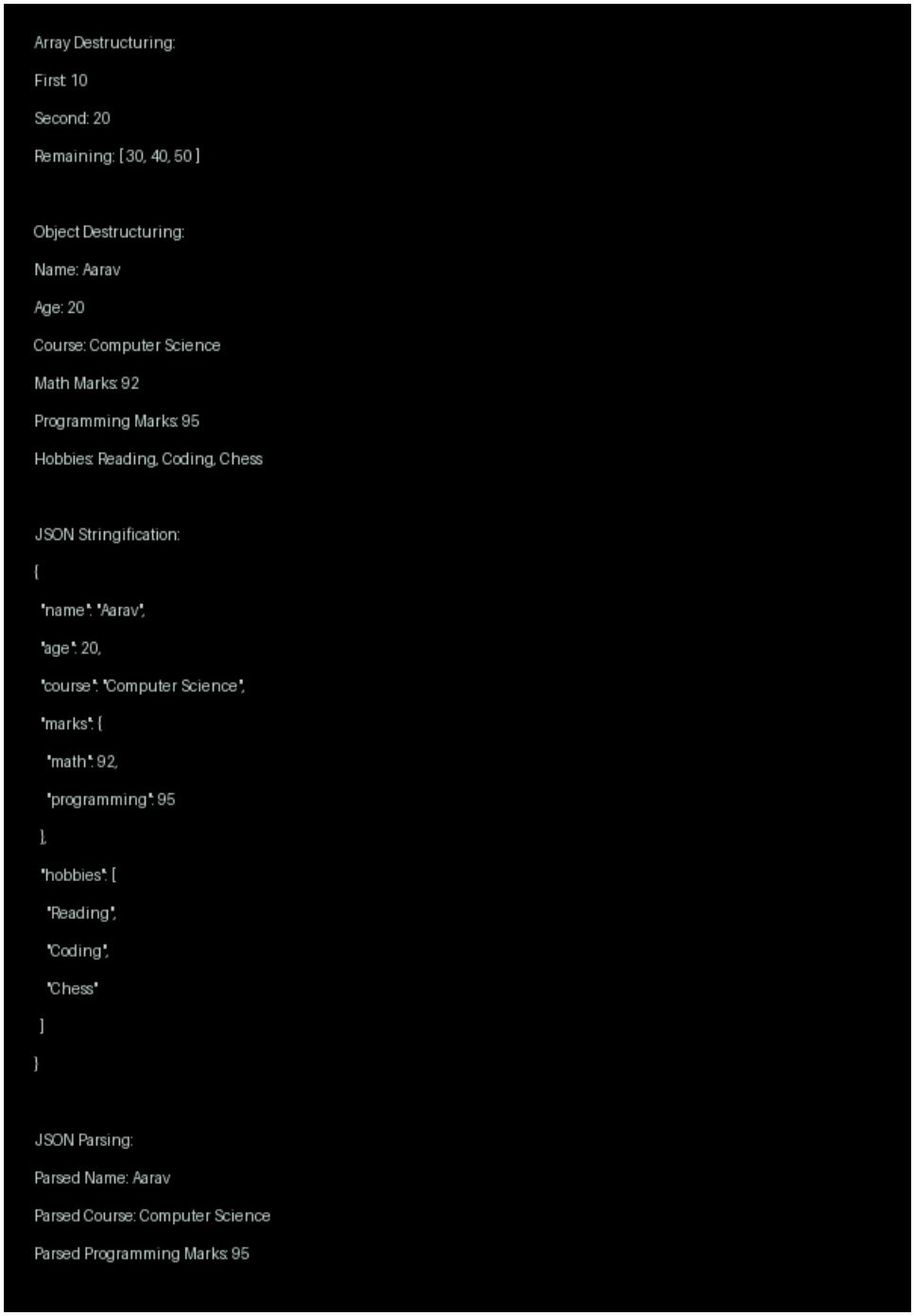

Figure 14 - Destructuring and JSON demo

Experiment No. 15

Aim - To write a comprehensive JavaScript mini project for building FlashMind, a frontend-only flashcard study application that allows students to create decks, add flashcards, generate flashcards from a topic using an API, study through interactive card flipping, store data using localStorage, and manage learning progress through DOM manipulation and event handling.

Concept Used - This mini project demonstrates the use of DOM manipulation to dynamically create and update flashcard decks, cards, study screens, and feedback messages. Event handling is used for actions such as adding cards, generating flashcards, flipping cards, marking answers as correct or wrong, and navigating between cards.

The project uses localStorage to save decks, flashcards, and progress so that data remains available even after refreshing the browser. The Fetch API is used to connect with an external AI endpoint such as Groq Llama 3.1 or another compatible model API to generate flashcards from a topic or source text. If the API is unavailable, the app can still create basic flashcards using a fallback method.

Code:

```
<!DOCTYPE html>

<html>

<head>

<title>FlashMind Mini</title>

<style>

body{font-family:Arial;background:#f7fbff;margin:0;padding:20px;color:#172033}

.app{max-width:800px;margin:auto}

h1{color:#0077cc}

input,textarea,button{width:100%;padding:10px;margin:6px 0;border-
radius:8px;border:1px solid #c7d8ea;font-size:14px}

button{background:#00a3ff;color:white;border:0;cursor:pointer;font-weight:bold}
```

```
.box,.item{background:white;padding:14px;margin:10px 0;border-radius:12px;box-shadow:0 8px 20px #d9ecff}

.flashcard{min-height:150px;display:flex;align-items:center;justify-content:center;text-align:center;border:2px solid #b7e2ff;border-radius:14px;padding:20px;font-size:20px;cursor:pointer}

.ok{background:#dcfce7;color:#166534;padding:10px;border-radius:8px;text-align:center}

.bad{background:#fee2e2;color:#991b1b;padding:10px;border-radius:8px;text-align:center}

</style>

</head>

<body>

<div class="app">

<h1>FlashMind Mini</h1>

<p>Create, generate and study flashcards.</p>

<div class="box">

<h2>Add Card</h2>

<input id="q" placeholder="Question">

<textarea id="a" placeholder="Answer"></textarea>

<button onclick="addCard()">Add</button>

</div>

<div class="box">

<h2>AI Generator</h2>

<input id="topic" placeholder="Topic">

<textarea id="src" placeholder="Source notes"></textarea>

<input id="key" placeholder="API key">

<button onclick="genCards()">Generate</button>

</div>

<div class="box">
```

<h2>Study</h2>

No card

<button onclick="known()">Got It</button>

<button onclick="wrong()">Don't Know</button>

<button onclick="next()">Next</button>

<h2>Saved Cards</h2>

<script>

let cards=JSON.parse(localStorage.getItem("flashmind_cards"))||[],i=0,ans=false;

function save(){localStorage.setItem("flashmind_cards",JSON.stringify(cards))}

function addCard(){

let
q=document.getElementById("q").value.trim(),a=document.getElementById("a").value.tr
im();

if(!q||!a)return alert("Enter question and answer");

cards.push({q,a,ok:0,bad:0});save();render();show();

document.getElementById("q").value="";document.getElementById("a").value="";}

async function genCards(){

let
t=document.getElementById("topic").value.trim(),s=document.getElementById("src").va
lue.trim(),k=document.getElementById("key").value.trim();

if(!t)return alert("Enter topic");

if(!k){cards.push({q:"What is "+t+"?",a:t+" is an important study
concept.",ok:0,bad:0},{q:"Why study "+t+"?",a:"It improves understanding of the
topic.",ok:0,bad:0});save();render();show();return}

let r=await
fetch("https://api.groq.com/openai/v1/chat/completions",{method:"POST",headers:{"Con
tent-Type":"application/json","Authorization":"Bearer

```
"+k},body:JSON.stringify({model:"llama-3.1-8b-
instant",messages:[{role:"user",content:"Create 3 flashcards as JSON array with q and a
for "+t+". Notes:"+s}]})});

let d=await r.json(),g=JSON.parse(d.choices[0].message.content);

g.forEach(c=>cards.push({q:c.q,a:c.a,ok:0,bad:0}));save();render();show();}

function render(){

let l=document.getElementById("list");l.innerHTML="";

cards.forEach((c,n)=>{l.innerHTML+=`<div
class="item"><b>Q:</b>${c.q}<br><b>A:</b>${c.a}<br>Known:${c.ok}
Wrong:${c.bad}<button onclick="del(${n})">Delete</button></div>`})}

function show(){

let
c=document.getElementById("card");document.getElementById("msg").innerHTML="";

if(!cards.length){c.innerText="No card";return}

ans=false;c.innerText=cards[i].q;}

function flip(){

if(!cards.length)return;

ans=!ans;document.getElementById("card").innerText=ans?cards[i].a:cards[i].q;}

function known(){

if(!cards.length)return;

cards[i].ok++;save();document.getElementById("msg").innerHTML='<div
class="ok">Got it!</div>';render();}


function wrong(){

if(!cards.length)return;

cards[i].bad++;save();document.getElementById("msg").innerHTML='<div
class="bad">You will get it next time.</div>';render();}

function next(){

if(!cards.length)return;

i=(i+1)%cards.length;show();}
```

```
function del(n){
cards.splice(n,1);i=0;save();render();show();}
render();show();
</script>
</body>
</html>
```

Output:

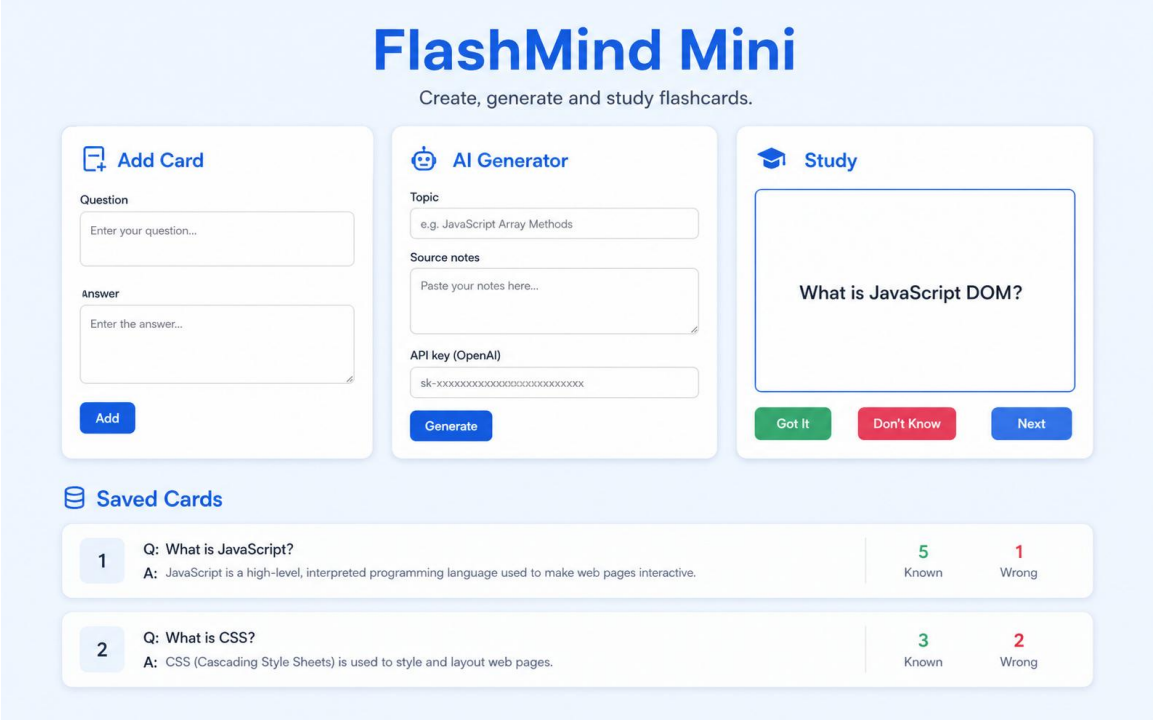

Figure 15: Output of FlashMind Mini showing flashcard creation, AI generation, study mode, answer feedback, and saved card progress using JavaScript, DOM manipulation, Fetch API, and localStorage.